Nama: Mikail Achmad

NIM: 24/542370/PA/23026

Kelas: KOM - B

Regular Class - Poem Generation Task

Link Google Colab Code:
https://colab.research.google.com/drive/1FiC5X0R3PcRUCnYL4Y0fLcMNaU9-KzqL?usp=sharing

1. Deskripsi Tugas

Tugas ini mengimplementasikan model Recurrent Neural Network (RNN) untuk next-word prediction pada korpus Shakespeare's Sonnets (154 soneta), lalu menggunakan model terlatih untuk men-generate puisi baru kata demi kata. Seluruh sel RNN (forward pass) ditulis menggunakan nn.Parameter kemudian divalidasi agar hasilnya identik dengan nn.RNN resmi.

2. Konfigurasi (diturunkan dari NIM 23026)

Parameter	Nilai	Keterangan
seq_len	20	Panjang window konteks (jumlah kata) yang dibaca RNN per langkah training
hidden	128	Ukuran hidden state RNN
emb	64	Ukuran embedding kata (fixed)
lr	0.003	Learning rate optimizer Adam
batch	64	Ukuran mini-batch (fixed)
epochs	8	Jumlah epoch training (fixed)
test_start	32	8 soneta berurutan mulai indeks 32 dijadikan test set
prompt	['o', 'that', 'time']	Kata awal untuk proses generate puisi
seed	3026	Seed untuk inisialisasi bobot, shuffling, dan sampling

3. Tahapan Implementasi

3.1 Tokenisasi & Vocabulary

Teks diubah menjadi huruf kecil, lalu dipecah menjadi token kata dan token tanda baca secara terpisah, dengan membuang karakter lain. Vocabulary dibangun dari soneta training, dengan kata yang muncul minimal dua kali masuk vocabulary dan sisanya menjadi <unk>.

Hasilnya diperoleh V = 1330 kata unik.

3.2 Encoding & Context Window

Setiap token diubah menjadi id integer. Window (X, Y) dibentuk dengan panjang L = 20: X adalah 20 id kata berurutan, dan Y adalah window yang sama digeser satu posisi ke kanan (target next-word).

Diperoleh 19.288 window data training dan 1.036 window data test.

3.3 RNN Manual (MyRNN)

Recurrence h_t = tanh(e_t . Wxh + h_(t-1) . Whh + bh) diimplementasikan dengan parameter Wxh (E,H), Whh (H,H), dan bh (H,) sebagai nn.Parameter. Implementasi ini berhasil divalidasi dengan menghasilkan output identik dengan nn.RNN resmi PyTorch ketika diberi bobot yang sama.

3.4 Training, Evaluasi, dan Baseline

Fungsi run_epoch melakukan shuffle + mini-batch + backpropagation + gradient clipping (norm 1.0) saat mode training, dan forward pass tanpa gradien saat mode evaluasi, lalu mengembalikan rata-rata cross-entropy per kata.

Sebagai pembanding, dihitung unigram baseline dengan add-one smoothing: p(w) = (count_train(w) + 1) / (N_train + V).

3.5 Generate Puisi

Model mengenerate kata baru satu per satu: logit kata terakhir dibagi temperature, logit <unk> dipaksa menjadi -1e9 (tidak pernah dipilih), lalu satu kata disampling dengan torch.multinomial dan diumpankan balik bersama hidden state untuk langkah berikutnya. Token <eos> dicetak sebagai baris baru.

4. Hasil Evaluasi

Sanity check sebelum training (CHECK 5):

Besaran	Prediksi (jurnal)	Hasil Aktual
Untrained test loss	7.000	7.252
ln(V)	7.000	7.193

Prediksi di jurnal sudah mendekati, model yang belum dilatih diharapkan berperilaku seperti distribusi uniform atas V kata, sehingga loss mendekati ln(V). Selisih hasil aktual (7.252 vs 7.193 = 0.059) di bawah batas toleransi (0.7), menandakan implementasi reshape logits/target untuk cross-entropy sudah benar.

Hasil training RNN manual (8 epoch):

Metode	Test Perplexity Terbaik	Epoch Terbaik	Test Perplexity Epoch Terakhir
Unigram baseline (add-one smoothing)	176.7	-	-
RNN manual	119.5	2	125.4

RNN manual mengalahkan baseline unigram (119.5 vs 176.7), membuktikan model memanfaatkan konteks kata sebelumnya. Dari epoch 2 ke epoch 8, train perplexity terus turun (106.3 -> 53.8) sementara test perplexity justru naik (119.5 -> 125.4). Model mulai overfitting pada korpus latihan yang kecil (146 soneta), sehingga bobot terbaik yang dipakai untuk evaluasi akhir adalah bobot di epoch 2, bukan bobot epoch terakhir.

Contoh hasil generate (prompt 'o that time', temperature=1.0):

"o that time things my then, some verse youth shall in that respect doth stand, and it might still you no how time lovest; your love is me, but as if new could be my proud in mistress ride. but, home see me for you. ah so is i mine forsake best is they"

5. Eksperimen Part C: Prediksi vs Hasil Aktual

5.1 C1 | Pengaruh Temperature

Temperature	Prediksi jumlah kata unik (dari 50)	Hasil aktual jumlah kata unik (dari 50)
T = 0.5	30	29
T = 1.0	35	37
T = 1.5	45	48

Prediksi mendekati hasil aktual dengan selisih 1-3 kata unik) yang membuktikan temperature rendah (0.5) membuat distribusi probabilitas lebih tajam sehingga output lebih repetitif, sedangkan temperature tinggi (1.5) meratakan distribusi probabilitas sehingga beberapa kata lebih jarang terpilih, membuat output lebih beragam namun kurang koheren.

5.2 C2 | Epoch Terbaik (Kapan Berhenti)

	Prediksi	Hasil Aktual
Epoch dengan test perplexity terendah	Epoch 3	Epoch 2

Prediksi hanya meleset satu epoch dari hasil aktual yang menunjukkan intuisi titik overfitting terjadi di awal training karena ukuran korpus kecil. Setelah epoch 2, train perplexity terus membaik tetapi test perplexity memburuk, mengonfirmasi bahwa epoch terakhir (ke 8) bukan pilihan terbaik. Itulah sebabnya notebook menyimpan bobot terbaik (best_state) berdasarkan test perplexity, bukan memakai bobot dari epoch terakhir.

5.3 C3 | Perbandingan dengan nn.LSTM (opsional)

	Prediksi jumlah parameter	Hasil aktual jumlah parameter
RNN manual	~280.000	281.394
nn.LSTM	~350.000	356.018

Model	Jumlah Parameter	Epoch Terbaik	Test Perplexity Terbaik
RNN manual	281.394	2	119.5
nn.LSTM	356.018	2	112.9
Unigram baseline	-	-	176.7

Prediksi jumlah parameter menunjukkan selisih di bawah 2%. Dari sisi performa, nn.LSTM mencapai test perplexity terbaik yang lebih rendah dibanding RNN manual (112.9 vs 119.5) dengan jumlah parameter sekitar 27% lebih banyak (empat gate pada LSTM vs satu recurrence sederhana pada RNN vanilla). Kedua model tetap mengalami overfitting setelah epoch 2 (test perplexity LSTM naik lagi dari 112.9 menjadi 135.4 di epoch 8). Ini menunjukkan bahwa dengan 146 soneta data training, pemilihan epoch berhenti sama pentingnya dengan pemilihan arsitektur.

6. Kesimpulan

- RNN manual (hand-written, tanpa nn.RNN) berhasil diimplementasikan dan divalidasi identik dengan nn.RNN resmi PyTorch, lalu dilatih sampai mengalahkan unigram baseline (119.5 vs 176.7 test perplexity) yang membuktikan model memanfaatkan konteks kata, bukan sekadar frekuensi kata.
- Korpus yang kecil (146 soneta training) membuat model cepat overfitting. Performa terbaik di test set justru tercapai pada epoch awal (epoch 2 dari 8), bukan di akhir training.
- Temperature pada tahap generate mengatur trade off antara koherensi (temperature rendah, output repetitif) dan keberagaman (temperature tinggi, output lebih acak), yang sesuai prediksi awal di jurnal.
- nn.LSTM memberi hasil sedikit lebih baik dibanding RNN manual, dengan biaya parameter yang lebih besar, namun kedua arsitektur sama-sama terbatas oleh ukuran data yang kecil.